

Homework 3

TAs: Siyuan, Yuteng, and Aditya

10-405/605/805: Machine Learning with Large Datasets

Due Thursday, October 1st at 11:59 PM Eastern Time

Submit your solutions via Gradescope.

IMPORTANT: Be sure to highlight where your solutions are for each question when submitting to Gradescope otherwise you will be marked 0 and will need to submit regrade request for every solution un-highlighted in order for fix it!

Note that Homework 3 consists of two parts: this written assignment and a programming assignment. Remember to fill out the collaboration section found at the end of this homework as per the course policy.

All students are required to complete the following:

1. Written Section 1.1 *[25 points]*
2. Programming Section 2.1 *[5 points]*
3. Programming Section 2.2 *[20 points]*

All students are required to complete **all** sections of the homework. Your homework score will be calculated as a percentage over the maximum points you can earn, which is 50.

Programming: The programming in this homework is **NOT** autograded however you are required to upload your completed `train.distributed.py` file to Gradescope, otherwise you will not receive credit for the programming sections.

1 Written Section [25 Points]

1.1 An Unbiased Sketch (25 pts)

The count-min sketch presented in lecture is biased, as the estimated count $\hat{c}_s$ for item $s \in \{1, \dots, k\}$ is always higher than (or equal to) the true count c_s . In this problem, we will explore an *unbiased* sketch.

Let g be a hash function chosen from a family G of independent hashes, such that g maps each item s to either $+1$ or -1 with equal probability:

$$P(g(s) = +1) = P(g(s) = -1) = 1/2.$$

Let's start with a simple version of the sketch, which uses one additional hash function, h , which maps each item s to one of m buckets. Assume that g and h are independent. When we observe an item s in the sequence, we simply update the sketch counter, C , as:

$$C[h(s)] = C[h(s)] + g(s).$$

Then, if we would like to predict the count for item s , we return:

$$\hat{c}_s = C[h(s)] g(s).$$

Given this sketch, please answer the following questions:

- a) *[10 points]* First, show that this simple sketch is unbiased, i.e., $\mathbb{E}[\hat{c}_s] = c_s$. (*Hint: Find an expression for $\hat{c}_s$ in terms of $g(s)$ and c_s , and use linearity of expectation.*)

- b) *[7 points]* We will now bound the probability of getting a bad estimate $\hat{c}_s$, i.e., one that differs substantially from the true count c_s . Show that:

$$P(|\hat{c}_s - c_s| \geq \epsilon \|\mathbf{c}_{-s}\|_2) \leq \frac{1}{\epsilon^2 m},$$

where $\mathbf{c}_{-s}$ is a vector of length $(k - 1)$ containing the counts for all items except s . You'll need the variance of the sketch which is given by

$$\text{Var}(\hat{c}_s) = \frac{1}{m} \sum_{j \in \{1, \dots, k\}, j \neq s} c_j^2$$

where m is the number of buckets for the hash function h .

(Hint: Consider Chebyshev's inequality: $P(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}$.)

- c) Similar to count-min sketch, we can improve the results from (c) by using r hash functions for mapping the items to buckets $(h_1, \dots, h_r)$, and r hash functions for assigning signs $(g_1, \dots, g_r)$. In particular, setting $m > \frac{3}{\epsilon^2}$ and $r > (\log(1/\delta))$, we can guarantee:

$$P(|\hat{c}_s - c_s| \geq \epsilon \|\mathbf{c}_{-s}\|_2) \leq \delta.$$

Note that while this result appears similar to the one we had for count-min sketch, it requires more buckets, m . Recall that for count-min sketch with $m > \frac{2}{\epsilon}$ and $r > (\log(1/\delta))$, we had the bound:

$$P(|\hat{c}_s - c_s| \geq \epsilon \|\mathbf{c}\|_1) \leq \delta.$$

With these results in mind, answer the questions below.

- i. *[4 points]* Explain a scenario where count-min sketch would be preferable to the unbiased sketch we derived (using the above bounds).

- ii. *[4 points]* Explain a scenario where the unbiased sketch we derived would be preferable to count-min sketch (using the above bounds).

2 Programming [25 Points]

2.1 Tuning Locality Sensitive Hashing [5 Points]

In this part of the programming assignment, you will (briefly) revisit the problem of document similarity. As we saw in lecture, locality sensitive hashing (LSH) is a scalable alternative to naive $O(n^2)$ pairwise document similarity calculation.

In this section, we will perform an empirical analysis of LSH parameter tuning. Recall that the likelihood of any two documents being hashed to the same bucket is given by

$$1 - (1 - \text{sim}(q, q')^r)^b$$

where $\text{sim}(q, q')$ is the similarity between two documents, b is the number of bands, and r is the number of rows per band.

Please navigate to <https://10605.github.io/vis/lsh-tuning.html> and find a visualization of this equation to answer the questions below.

2.1.1 Empirical Questions *[5 points]*

- (a) *[1 points]* How does the number of bands affect the precision and number of pairs found?

- (b) *[2 points]* What is the trade-off between the number of hash functions and accuracy?

- (c) *[2 points]* How does changing the number of hash functions affect the S-shaped curve?

2.2 Synchronous Data-Parallel Training [20 points]

2.2.1 Introduction

In this section, you will implement synchronous, CPU-based data-parallel training for the binary classification model from Homework 2. Each worker trains a copy of the same multilayer perceptron on its own shard of the training data. Before each optimizer step, all workers average their gradients so that their models remain synchronized. You will also implement model averaging, which synchronizes parameter values across workers. Data partitioning with a shuffled `DistributedSampler` is provided in the training loop. You will measure how the number of workers affects validation loss over training time and total training throughput. Implement the communication yourself using `torch.distributed`; do not replace the four functions with PyTorch's `DistributedDataParallel` wrapper.

The release contains the following files:

- `train_distributed.py` (where you implement the TODOs)
- `features.csv` (the supplied dataset)
- `hw3.coding.ipynb` (public checks, runs, and plotting code)

2.2.2 Getting Started

1. **Set up Python.** Use an environment with PyTorch, Polars, NumPy, Jupyter, and Matplotlib. If necessary, run `python -m pip install torch polars numpy jupyter matplotlib`. Open the notebook from the extracted assignment directory using that environment.
2. **Write your code.** Complete the four TODO functions in `train_distributed.py`. Retain their names, signatures, and the provided training loop. The notebook contains no implementation work for students.
3. **Check your implementation.** Restart the notebook kernel and run it from top to bottom. **Part A contains four public correctness tests**, one for each TODO in `train_distributed.py`. Passing these tests does not guarantee correctness when running the distributed experiments.
4. **Run the experiments.** **Part B is provided experiment and plotting code**; you will not implement anything in this section. It will launch, run, and plot results for the experiments you are asked to complete.
5. **Complete the empirical questions.** After completing the TODOs and plotting your experimental results you should return to this handout to answer the empirical questions.
6. **Submit your code to Gradescope.** Finally, submit your `train_distributed.py` file to Gradescope!

2.2.3 Instructions

1. **Initialize the process group.** In `initialize_distributed()`, join a Gloo process group using the environment supplied by `torchrun`. Return the process rank and number of workers. The [PyTorch torchrun guide](#) and [process-group API documentation](#) provide the needed calls.
2. **Start from the same model.** In `broadcast_parameters()`, broadcast every parameter from rank 0, in the same order on every worker. Modify parameter values without recording the operations in autograd. See the [broadcast documentation](#) and [no-grad documentation](#).
3. **Average packed gradients.** In `average_gradients()`, use exactly one `all_reduce` per step, irrespective of the number of model parameters. All workers must call the same collective on matching tensor shapes. For a one-worker run, leave the gradients as they are and skip communication. Use the pseudocode below; the [torch.cat](#) and [all-reduce](#) references describe the relevant API calls.

Algorithm 1 – Average packed gradients

```

1: Input: parameters  $(\theta_1, \dots, \theta_P)$ , each with a gradient; worker count  $n$ 
2: if  $n = 1$  then return
3: Disable autograd tracking for the remaining operations
4: for  $i = 1, \dots, P$  do
5:    $g_i \leftarrow \theta_i.\text{grad}$ ;  $s_i \leftarrow \text{shape}(g_i)$ ;  $m_i \leftarrow \text{numel}(g_i)$ 
6: end for
7:  $v \leftarrow \text{concat}(\text{flatten}(g_1), \dots, \text{flatten}(g_P))$ 
8:  $v \leftarrow \text{allreduces}_{\text{SUM}}(v)$  (one collective across all workers)
9:  $v \leftarrow v/n$ ;  $o \leftarrow 0$ 
10: for  $i = 1, \dots, P$  do
11:   copy_into( $g_i$ , reshape( $v[o : o + m_i]$ ,  $s_i$ ))
12:    $o \leftarrow o + m_i$ 
13: end for

```

4. **Average packed model parameters.** In `average_model()`, use exactly one `all_reduce` per call, irrespective of the number of model parameters. All workers must pack parameters in the same order and call the same collective on matching tensor shapes. For a one-worker run, leave the parameters as they are and skip communication. Modify parameter values in place without recording the operations in autograd, preserving the parameter objects used by the optimizer. Follow Algorithm 2 below, using the same `torch.cat` and `all-reduce` API calls as in gradient averaging.

Algorithm 2 – Average packed model parameters

```

1: Input: parameters  $(\theta_1, \dots, \theta_P)$ ; worker count  $n$ 
2: if  $n = 1$  then return
3: Disable autograd tracking for the remaining operations
4: for  $i = 1, \dots, P$  do
5:    $s_i \leftarrow \text{shape}(\theta_i)$ ;  $m_i \leftarrow \text{numel}(\theta_i)$ 
6: end for
7:  $v \leftarrow \text{concat}(\text{flatten}(\theta_1), \dots, \text{flatten}(\theta_P))$ 
8:  $v \leftarrow \text{allreduces}_{\text{SUM}}(v)$  (one collective across all workers)
9:  $v \leftarrow v/n$ ;  $o \leftarrow 0$ 
10: for  $i = 1, \dots, P$  do
11:   copy_into( $\theta_i$ , reshape( $v[o : o + m_i]$ ,  $s_i$ ))
12:    $o \leftarrow o + m_i$ 
13: end for

```

2.2.4 Experiments

The first experiment runs data parallel training using 1, 2, and 4 workers. The left plot shows validation binary cross-entropy against cumulative *training* time. The right shows examples processed across all workers divided by training time. Each worker uses batches of 512, so the effective batch grows with worker count; learning rates are supplied for 1, 2, and 4 workers.

[7.5 points] Insert here your two plots of validation loss and throughput for experiment 1

A large, empty rectangular box with a thin black border, intended for the student to insert their plots for experiment 1.

The second experiment runs periodic parameter mixing training with 1, 4, and 16 intervals on 4 workers. In this training setup, each worker updates parameter locally, and model parameters are averaged every several optimization steps. The left panel shows the validation loss vs. training time for different interval choices, and the right panel shows the training throughput. Less frequent averaging reduces communication frequency and may improve throughput.

[7.5 points] Insert here your two plots of validation loss and throughput for experiment 2

A large, empty rectangular box with a thin black border, intended for the student to insert their plots for experiment 2.

2.2.5 Empirical Questions [5 points]

- (a) [1 points] **Select all that apply:** Which statements can explain why adding more workers gives only a small speedup when training a small model?
- ☐ Communication can take as much time as the model's computation.
 - ☐ Adding workers always makes each training iteration faster.
 - ☐ Workers may spend time waiting for each other during synchronization.
 - ☐ Adding workers removes the need to communicate.
 - ☐ None of the above.
- (b) [1 points] **Select one:** Each worker uses a batch size of 512. When moving from one worker to four workers with gradient averaging, what happens to the global batch size and the number of optimizer steps per worker per epoch? Assume the dataset size stays fixed and ignore incomplete batches and padding.
- ☐ The global batch size stays at 512; each worker takes one quarter as many steps.
 - ☐ The global batch size becomes 2048; each worker takes the same number of steps.
 - ☐ The global batch size becomes 2048; each worker takes one quarter as many steps.
- (c) [2 points] A run with eight workers processes more training examples per second than a run with two workers. Does this guarantee that it reaches validation loss 0.52 sooner? Explain briefly.

- (d) [1 points] **Select all that apply:** Keep the worker count, local batch size, and total number of local optimizer steps fixed. What changes when the model averaging period increases from $K = 1$ to $K = 16$?
- ☐ Scheduled training-model averaging happens about 16 times less often.
 - ☐ Each worker's local batch size becomes 16 times larger.
 - ☐ Workers can take more local steps before their model parameters are averaged.
 - ☐ The final validation loss is guaranteed to decrease.
 - ☐ None of the above.

3 Collaboration Questions

1. (a) Did you receive any help whatsoever from anyone in solving this assignment?

(b) If you answered 'yes', give full details (e.g. "Jane Doe explained to me what is asked in Question 3.4")

2. (a) Did you give any help whatsoever to anyone in solving this assignment?

(b) If you answered 'yes', give full details (e.g. "I pointed Joe Smith to section 2.3 since he didn't know how to proceed with Question 2")

3. (a) Did you find or come across code that implements any part of this assignment?

(b) If you answered 'yes', give full details (book & page, URL & location within the page, etc.).